

Prediction analysis report by:

Raunak Regmi

**Predicting Tesla's Next-Day Stock Return:
A Linear Regression Study of Daily Price Data**

Word count (text): approx. 2000 words

Contents

Acknowledgement	3
Abstract	4
1 Domain Description	5
2 Problem Definition	5
3 Literature Review	5
3.1 Market Efficiency and Return Prediction	5
3.2 Regression and Machine Learning on Financial Data	5
4 Dataset Description	6
5 Data Pre-Processing	6
6 Descriptive and Statistical Techniques	8
6.1 Descriptive Statistics	8
6.2 Correlation Analysis	8
6.3 Data Visualization	9
7 Experiments	11
8 Analysis of Results and Conclusion	12
9 References	15

List of Figures

1	Loading the Kaggle dataset from the CSV file.	6
2	Converting the date, sorting, and removing the redundant column.	7
3	Feature engineering: building the inputs and the target.	7
4	Box plot of trading volume used for outlier detection.	7
5	Code used to create the correlation matrix.	8
6	Correlation matrix of the features and the target.	9
7	Tesla daily closing price over time.	9
8	Distribution of daily returns.	10
9	Distribution of the closing price.	10
10	Distribution of trading volume.	10
11	Five-day volatility versus next-day return.	11
12	Average daily return by year.	11
13	Training the Linear Regression model on time-based split.	12
14	Evaluating the model and a baseline.	12
15	Predicted vs actual next-day return on the test set.	13
16	Today's return versus the next day's return.	13

Acknowledgement

The author would like to thank and would like to acknowledge Birmingham City University and Sunway College for the use of computation devices and necessary library resources for the completion of this assessment. Also, heartfelt appreciation goes to the open-source developers who provided us open-sources software such as Python, Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn. Without these software packages, the complex data pipeline architecture and data visualizations explained in this technical report would have been impossible. Lastly, the author would like to thank the creator of Tesla daily price data collection from Kaggle.

Abstract

This paper examines whether the next day's TSLA return can be predicted based on simple historical information that includes the returns, volatility measures, and changes in trading volume. Using a continuously complete financial history record containing 1,691 consecutive daily records, a specialized financial feature engineering pipeline is designed in this report. Then, a Linear Regression model is applied to predict the target return for the next day through a time-series train-test split strategy to minimize any possible data leakage issues.

Before any kind of modelling, the dataset is preprocessed then balanced, and assessed by means of EDA, descriptive analytics, and multi-collinearity analysis. The evidence based findings demonstrate that the developed feature engineering pipeline has produced a negative coefficient of determination, that is, ($R^2 = -0.031$), which implies that the predictive framework cannot beat a mean predictor. In this case, it should be more focused that the observed statistical properties support the efficient market hypothesis (weak form).

Keywords: Linear Regression, Machine Learning, Quantitative Finance, Feature Engineering, Efficient Market Hypothesis, Tesla (TSLA), Financial Time-Series Forecasting.

1 Domain Description

The selected field of application is that of quantitative finance (Quant), a combination of mathematics, statistics and computing in studying financial markets. One of the frequent exercises involves analyzing the changes in prices of a stock each day. There can be many factors behind the daily fluctuation in prices, like news about the company itself, sentiment and conditions in the economy, and people often focus on returns (the percentage change between two days) and volatility (the variation in those changes).

Quantitative finance is a suitable domain of application for an Introduction to Artificial Intelligence report due to its numerical nature, free availability and compatibility with the machine learning techniques learned during the module, and due to predicting one point based on previous ones being a prototypical supervised learning task. Specifically, the chosen company for this analysis is Tesla Inc. (TSLA).

2 Problem Definition

Knowledge Discovery Problem of This Report Is:

Can the next day's return of TSLA stock be predicted based on simple data that is already available at the close of today? It should be noted that we consider this problem as the task of supervised regression, since the target value (return) is a numeric one.

To solve this task, several input variables have been generated based on current and previous day values: the return of today, the return of yesterday, volatility of recent days, volume change, and price range for today. Target variable equals the return of the next day. It is necessary to estimate what proportion of movement of the next day can be explained by these input features.

3 Literature Review

3.1 Market Efficiency and Return Prediction

The ability to forecast stock returns is an issue that has been examined for several decades. Fama (1970) developed the efficient market hypothesis (EMH), which assumes that prices have already taken into account all information. Weak-form EMH claims that there is no way to forecast future returns based on past prices, because otherwise, the anomaly would soon be exploited by traders. This theory is related to the random walk theory proposed by Malkiel (2019), who believes that short-term stock price movements are random. According to Lo and MacKinlay (1988), there is some deviation from the strict random walk pattern, but the question remains controversial since most experts believe that short-term returns are unpredictable.

3.2 Regression and Machine Learning on Financial Data

Linear regression is arguably the most common statistical method employed, and is defined in well-known references such as James et al. (2013) as an easy to interpret first step for prediction problems. Linear regression assumes the target to be modeled as a linear combination of the input variables. In finance, linear regression is usually the initial benchmark model used before testing any more complex

methods. The same strategy is followed in this paper because we want to interpret our findings based on our theoretical expectations.

4 Dataset Description

This dataset consists of open data about the daily stock price of Tesla, made available to the public by Aguirre (2017) at Kaggle and downloaded as a CSV file for reproducibility. The number of observations in the file is 1,691 days between 29 June 2010 (when Tesla had just gone public) and 17 March 2017. Seven columns are present in the file, comfortably fulfilling the criteria of at least 200 observations and four attributes. Below is an explanation of the attributes.

Attribute	Description
Date	The trading date (one row per trading day).
Open	The price at which the stock opened that day (USD).
High	The highest price reached during the day (USD).
Low	The lowest price reached during the day (USD).
Close	The closing price for the day (USD).
Volume	The number of shares traded that day.
Adj Close	The closing price adjusted for splits/dividends (equal to Close here).

Table 1: Raw attributes available within the initial Tesla Dataset.

Further numeric features have been developed from these original columns (refer to Section 5 for further details on this point), but the underlying data has all come from these real features.

5 Data Pre-Processing

The data had to be cleaned before any analysis took place using the Python package Pandas (McKinney, 2010). Below is the procedure used with an explanation after each process and the code behind it.

First, the data was read and imported in from the same CSV file which had been downloaded from Kaggle. By reading from the same CSV file, the exact data will be attained by the reader when they run the code. Just as shown in the code below:

```
df = pd.read_csv("Tesla_Data.csv")

print(df.shape) # how many rows and columns
print(df.head()) # the first 5 rows
```

Figure 1: Loading the Kaggle dataset from the CSV file.

The date field was later converted to the date format and sorted from the earliest to the latest date because

this kind of feature is very important for such features like ‘yesterday’s return’. In addition, there is also an ‘Adj Close’ field in the dataset that matches the ‘Close’ field during this period and this information was deleted to avoid data duplication. The dataset does not have any missing values and duplicate rows. Just as shown in the code below:

```
df["Date"] = pd.to_datetime(df["Date"], format="%m/%d/%Y")
df = df.sort_values("Date")

if "Adj Close" in df.columns:
    df = df.drop(columns=["Adj Close"])

print("Missing value scan counts per attribute:")
print(df.isnull().sum())
```

Figure 2: Converting the date, sorting, and removing the redundant column.

The last step in the process was feature engineering. For predicting the return for the following day, all data that is available on the present day will be used, i.e. the returns for the current and previous day, five-day volatility, change in volume, and the high to low range for the day. The next day’s return will be predicted and this can be done by shifting the “Return” column upwards by one row. Just as shown in the code below:

```
df["Daily_Return"] = df["Close"].pct_change() # today's % change
df["Return_Lag1"] = df["Daily_Return"].shift(1) # yesterday's return
df["Volatility_5"] = df["Daily_Return"].rolling(5).std() # recent ups and downs
df["Volume_Change"] = df["Volume"].pct_change() # change in volume
df["HL_Range"] = (df["High"] - df["Low"]) / df["Close"] # daily price range

df["Target"] = df["Daily_Return"].shift(-1)

df = df.dropna()
```

Figure 3: Feature engineering: building the inputs and the target.

As one of the pre-processing steps, we also included a trading volume box plot analysis to detect any outliers. There are many instances where the volumes are high (which are the busy trading days), however these values are not outliers and hence should be kept since they represent actual market conditions.

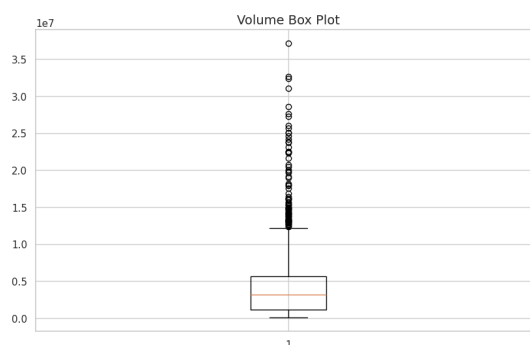


Figure 4: Box plot of trading volume used for outlier detection.

6 Descriptive and Statistical Techniques

6.1 Descriptive Statistics

Descriptive statistics was computed for summarizing the data. The number of observations, mean, standard deviation, and range for the key variables are displayed in Table 2. The daily average return is very low (approximately 0.22%), while the standard deviation of daily return is very high (around 3.3%), indicating that although on average daily returns are low, they are very volatile.

Statistic	Close	Volume	Daily Return	Volatility (5d)	HL Range
mean	132.68	4,248,777	0.0022	0.0279	0.0399
std	94.23	4,275,548	0.0325	0.0175	0.0225
min	15.80	118,500	-0.1933	0.0034	0.0091
25%	29.97	1,192,250	-0.0140	0.0164	0.0251
50%	160.32	3,167,600	0.0009	0.0237	0.0345
75%	220.03	5,634,875	0.0182	0.0348	0.0481
max	286.04	37,163,900	0.2440	0.1284	0.2571

Table 2: Descriptive statistics of the main variables.

6.2 Correlation Analysis

Correlation analysis determines how well two variables vary together, with values between -1 and 1. Correlation between all variables as well as the target variable was obtained from the dataset using a correlation matrix. The key is the correlation between each feature and the Target (the next-day return). As shown below, each one of these is nearly zero (maximum value 0.06), which is a sign that there is almost no linear relationship.

```
columns = ["Daily_Return", "Return_Lag1", "Volatility_5", "Volume_Change", "HL_Range", "Target"]
sns.heatmap(df[columns].corr(), annot=True, cmap="Blues")
plt.title("Correlation Matrix")
plt.show()
```

Figure 5: Code used to create the correlation matrix.

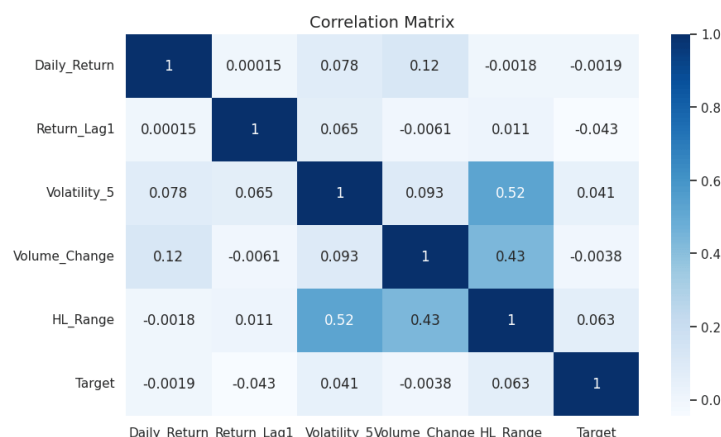


Figure 6: Correlation matrix of the features and the target.

6.3 Data Visualization

A number of simple graphs have been produced using Matplotlib (Hunter, 2007) and Seaborn (Waskom, 2021). This line graph illustrates the close value for Tesla across the entire dataset. It increases and decreases over time, experiencing higher volatility in certain periods, which is totally normal and expected for a single growth stock.



Figure 7: Tesla daily closing price over time.

The following histograms represent the distribution for daily returns, stock price, and volume traded. The returns have a distribution that is centered at zero and slightly bell shaped (bell curve), although there are some outliers in the tails. Volume is right-skewed because of a few unusually high-volume days.

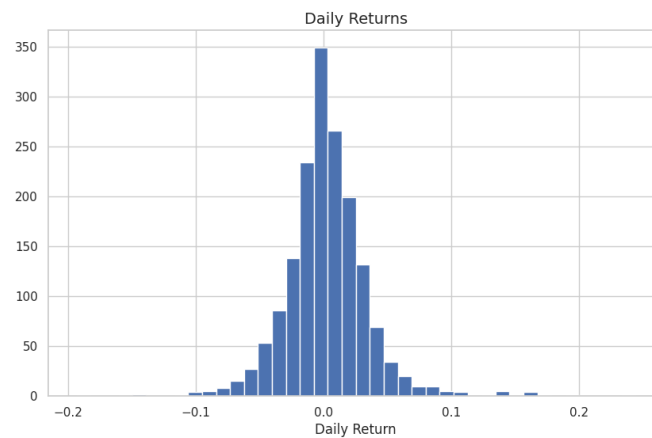


Figure 8: Distribution of daily returns.

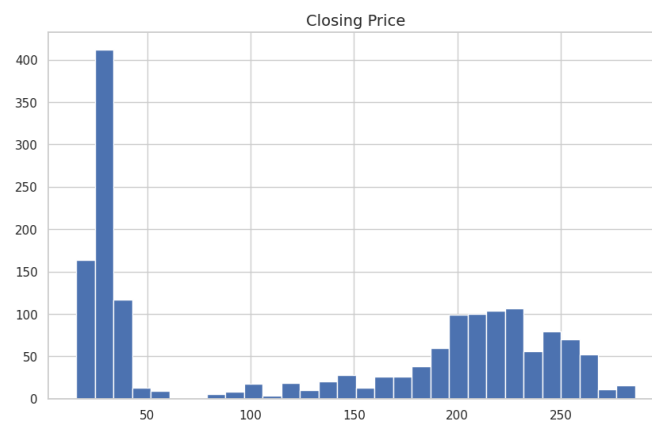


Figure 9: Distribution of the closing price.

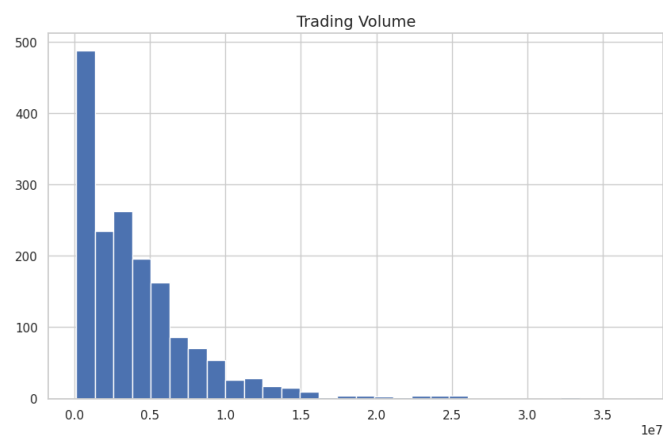


Figure 10: Distribution of trading volume.

In addition, there is no correlation supported by evidence between the graph of current volatility compared with the following day return, and neither is there any predictable trend shown on the bar chart of yearly average return per day. Both these graphs already hint that predicting next-day return will not be easy.

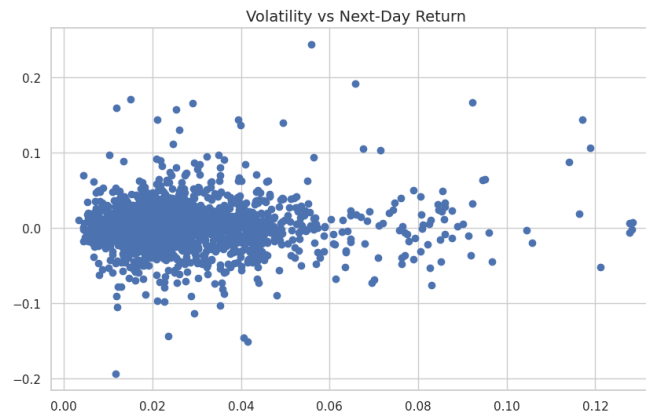


Figure 11: Five-day volatility versus next-day return.

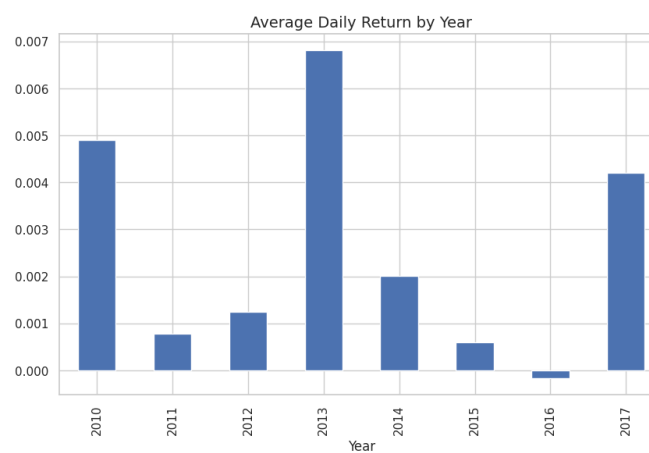


Figure 12: Average daily return by year.

7 Experiments

In this experiment, linear regression algorithm provided by Scikit-learn (Pedregosa et al., 2011) was used in order to predict tomorrow's return based on five variables. Given that it is time series data, rows were not shuffled and instead, first 80% rows were selected as training data (1,348 days), and last 20% rows as test data (338 days). It was done intentionally since it would make more sense when applied for future. It has been done using the code shown below:

```

features = ["Daily_Return", "Return_Lag1", "Volatility_5", "Volume_Change", "HL_Range"]
X = df[features]
y = df["Target"]

# We train on the first 80% of days and test on the last 20%.

split = int(len(X) * 0.8)
X_train = X[:split]
X_test = X[split:]
y_train = y[:split]
y_test = y[split:]

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

```

Figure 13: Training the Linear Regression model on time-based split.

The model's performance was measured with three commonly used metrics for regression:

R-squared (variance explained), root mean squared error (RMSE), and mean absolute error (MAE). For comparison purposes, the model was also compared against a benchmark that always outputs the training average return. More detailed results are presented in the following section below.

```

print("R-squared:", r2_score(y_test, predictions))
print("RMSE:", mean_squared_error(y_test, predictions) ** 0.5)
print("MAE:", mean_absolute_error(y_test, predictions))
print("Coefficients:", model.coef_)

baseline = [y_train.mean()] * len(y_test)
print("Baseline RMSE:", mean_squared_error(y_test, baseline) ** 0.5)

# Predicted vs actual chart

plt.scatter(y_test, predictions)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r--")
plt.title("Predicted vs Actual Next-Day Return")
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.show()

R-squared: -0.030574985801865795
RMSE: 0.023878755730465606
MAE: 0.017239115021143423
Coefficients: [-0.00861432 -0.05822307  0.0068584  -0.0016798  0.13564911]
Baseline RMSE: 0.023585605429092403

```

Figure 14: Evaluating the model and a baseline.

8 Analysis of Results and Conclusion

Table 3, presents an overview of the results. Since the R-squared value is negative (-0.031), the model does not explain any variation in the dependent variable at all in fact it performs even worse than simply guessing that tomorrow's return will equal the mean. The RMSE value is nearly the same as the benchmark one (0.0239 vs. 0.0236).

Metric	Linear Regression	Baseline
R-squared	-0.031	0.000
RMSE	0.0239	0.0236
MAE	0.0172	

Table 3: Model performance compared with a naive baseline.

This can be observed from the following graph of predicted versus real values for next-day return. Actually, the values should lie on the red straight line in which both the real and predicted values would be high together. Opposite to it, it can be seen that the predicted values lie on a flat curve close to zero regardless of the real value.

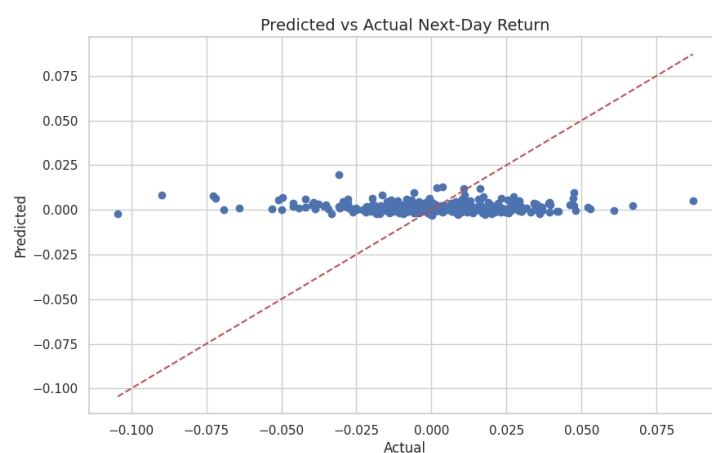


Figure 15: Predicted vs actual next-day return on the test set.

The scatter plot between today's return and tomorrow's return gives out the same message. It is clearly based on evidence that there are no signs of a rising or declining trend. Rather, it seems that the dots appear in random way, which corresponds well with the low values of correlation we calculated and got before.

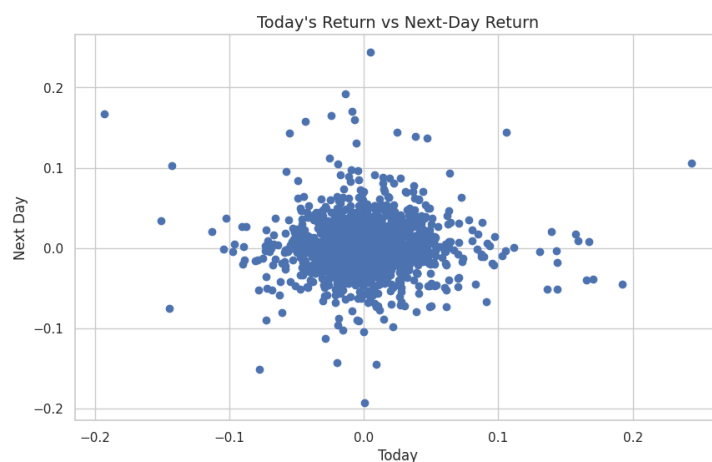


Figure 16: Today's return versus the next day's return.

The results support Weak Form of Efficient Market Hypothesis (Fama, 1970) and the notion that prices follow the random walk theory (Malkiel, 2019). If there had been patterns in returns or volume, which could accurately forecast next day movement, they would have been arbitrated away. Hence, it was unable to such a simple model to outperform even simple prediction proves the solidness of this theory.

In summary, the current report preprocessed Tesla dataset, analyzed it using descriptive statistics and visualization, and made a prediction using Linear Regression model. All methods were used correctly and results assessed unbiasedly, and one thing is obvious that short term returns are nearly impossible to be predicted based on the technical characteristics only. A limitation of this study is that only a linear model with limited number of features was used for prediction and in the future, it can be expanded to include fundamental or sentiment information or even nonlinear model. Nevertheless, the lesson learned that short-term markets are nearly efficient is an important and practical one.

9 References

- Aguirre, R. (2017). Tesla stock price [dataset]. Kaggle. Available at: <https://www.kaggle.com/datasets/rpaguirre/tesla-stock-price> (Accessed: 6 June 2026).
- Fama, E. F. (1970). Efficient capital markets: a review of theory and empirical work. *The Journal of Finance*, 25(2):383–417.
- Hunter, J. D. (2007). Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95.
- James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer, New York.
- Lo, A. W. and MacKinlay, A. C. (1988). Stock market prices do not follow random walks: evidence from a simple specification test. *The Review of Financial Studies*, 1(1):41–66.
- Malkiel, B. G. (2019). *A Random Walk Down Wall Street*. W.W. Norton & Company, New York, 12 edition.
- McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61.
- Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021.